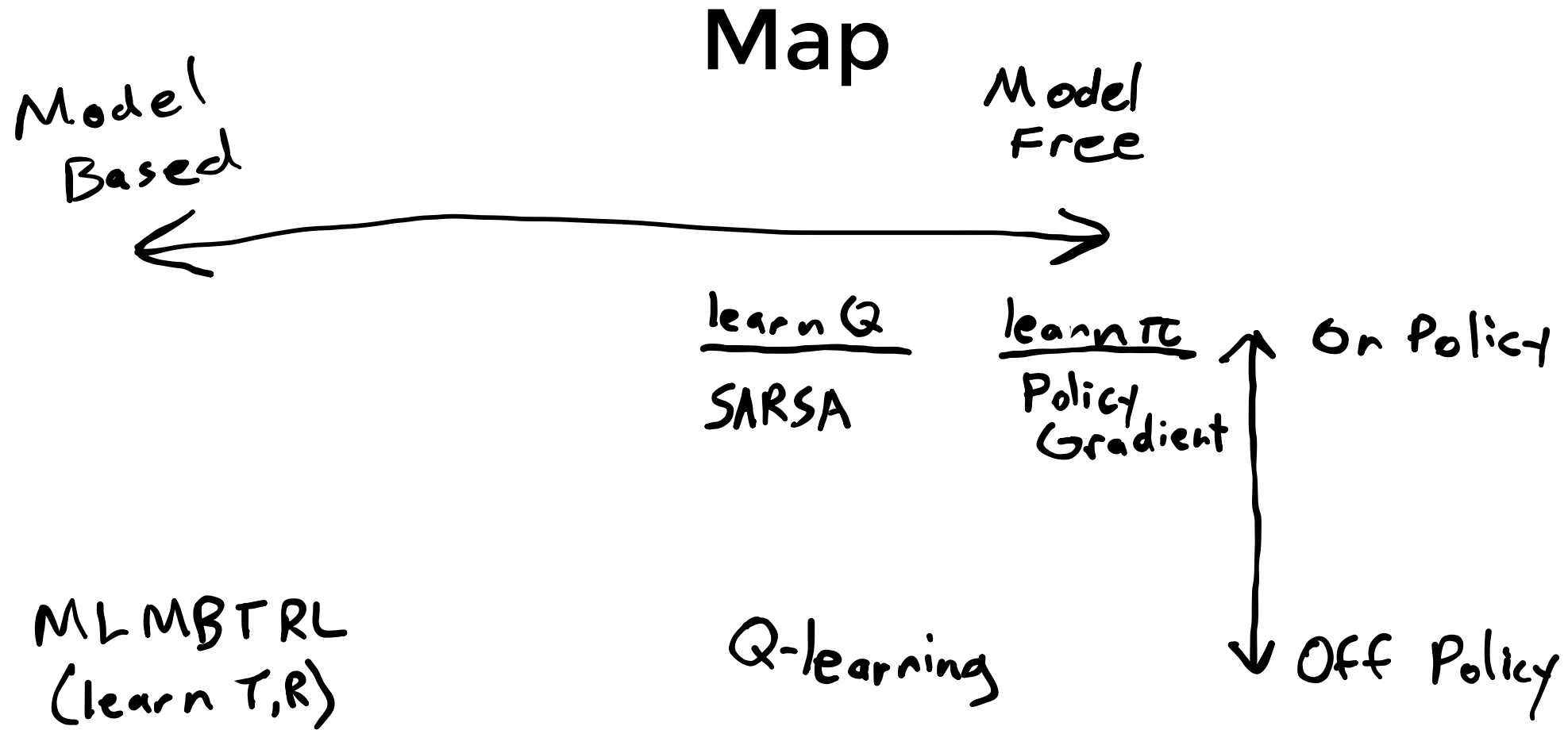


Actor-Critic
Reward Shaping
Advanced Exploration
Entropy Regularization



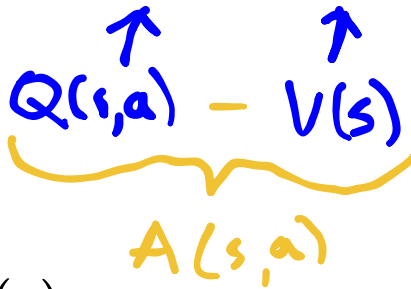
Challenges:

1. Exploration vs Exploitation ←
2. Credit Assignment
3. Generalization

1. Actor-Critic
2. Reward Shaping
3. Advanced Exploration
4. Entropy Regularization
5. Wisdom

1. Actor-Critic

$$\nabla U(\theta) = E_{\tau} \left[\sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k | s_k) \gamma^k (r_{k,\text{to-go}} - r_{\text{base}}(s_k)) \right]$$



Advantage Function: $A(s, a) = Q(s, a) - V(s)$

- Actor: π_{θ}
- Critic: Q_{ϕ} and/or A_{ϕ} and/or V_{ϕ}

Alternate between training Actor and Critic

Problem: Instability

Actor-Critic

Which should we learn? A , Q , or V ?

$$\nabla U(\theta) = E_{\tau} \left[\sum_{k=0}^d \nabla_{\theta} \log \pi_{\theta}(a_k | s_k) \underbrace{\gamma^k (r_k + \gamma V_{\phi}(s_{k+1}) - V_{\phi}(s_k))}_{\text{temporal difference residual}} \right]$$

$$l(\phi) = E \left[\left(V_{\phi}(s) - \hat{V}^{\pi_{\theta}}(s) \right)^2 \right]$$

↑ estimate with
reward to go
from sims

Generalized Advantage Estimation

$$A(s_k, a_k) \approx r_k + \gamma V_\phi(s_{k+1}) - V_\phi(s_k) \quad \leftarrow \text{High Bias}$$

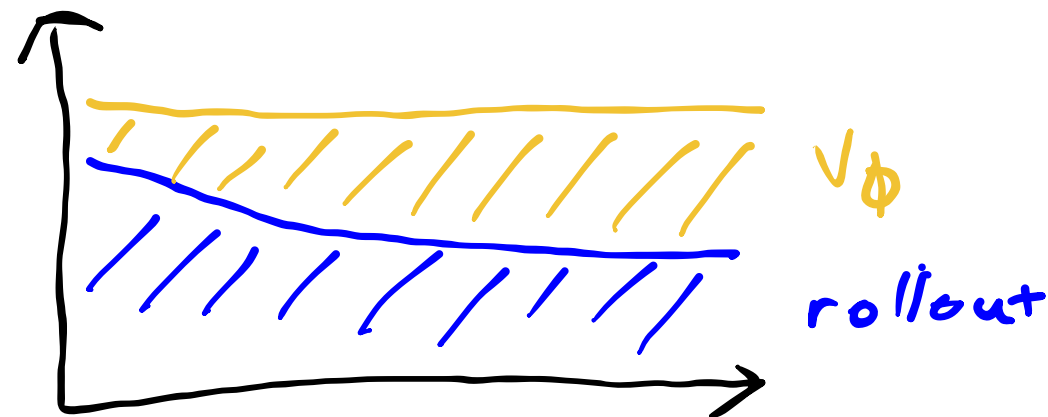
$$A(s_k, a_k) \approx \sum_{t=k}^{\infty} \gamma^{t-k} r_t \quad \leftarrow \text{High Variance}$$

$$A(s_k, a_k) \approx \left(\sum_{t=k}^{d-1} \gamma^{t-k} r_t \right) + \gamma^{d-k} r_d + \gamma V_\phi(s_{d+1}) - V_\phi(s_d)$$

$\leftarrow$ When should we stop?

$$\text{let } \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

$$A_{\text{GAE}}(s_k, a_k) \approx \sum_{t=k}^{\infty} (\gamma\lambda)^{t-k} \delta_t$$

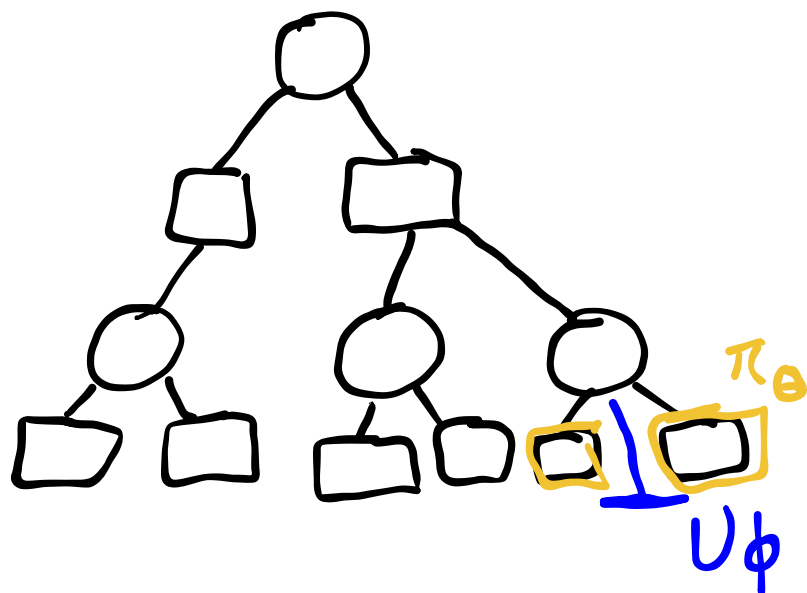


Alpha Zero: Actor Critic with MCTS

1. Use π_θ and U_ϕ in MCTS
2. Learn π_θ and U_ϕ from tree

$$\ell(\theta) = -\mathbb{E}_s \left[\sum_a \pi_{\text{MCTS}}(a | s) \log \pi_\theta(a | s) \right]$$

$$\pi_{\text{MCTS}}(a | s) \propto N(s, a)^\eta$$



$$\ell(\phi) = \frac{1}{2} \mathbb{E}_s \left[(U_\phi(s) - U_{\text{MCTS}}(s))^2 \right]$$

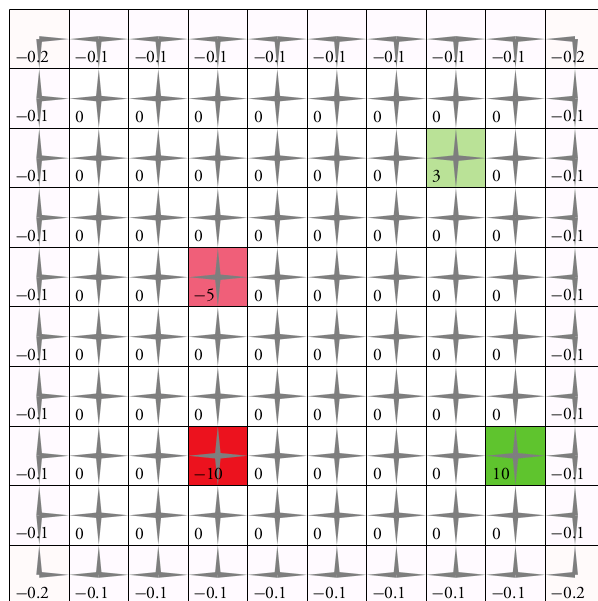
$$U_{\text{MCTS}}(s) = \max_a Q(s, a)$$

$$a = \arg \max_a Q(s, a) + c \pi_\theta(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

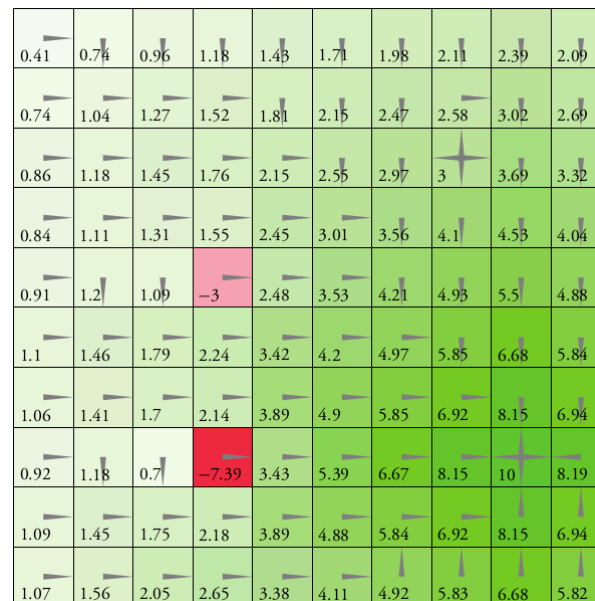
2. Reward Shaping

Which is easier to learn on?

Sparse Reward



Dense Reward



Coast Runners 7: A Cautionary Tale



CoastRunners 7

<https://www.youtube.com/watch?v=tlOlHko8ySg>

Reward Shaping

"As a general rule, it is better to design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave." - Stuart Russell

Reward

-0.2	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.2
-0.1	0	0	0	0	0	0	0	0	-0.1
-0.1	0	0	0	0	0	0	3	0	-0.1
-0.1	0	0	0	0	0	0	0	0	-0.1
-0.1	0	0	-5	0	0	0	0	0	-0.1
-0.1	0	0	0	0	0	0	0	0	-0.1
-0.1	0	0	0	0	0	0	0	0	-0.1
-0.1	0	0	-10	0	0	0	0	10	-0.1
-0.1	0	0	0	0	0	0	0	0	-0.1
-0.2	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.1	-0.2

Value

0.41	0.74	0.96	1.18	1.45	1.71	1.98	2.11	2.39	2.09
0.74	1.04	1.27	1.52	1.81	2.15	2.47	2.58	3.02	2.69
0.86	1.18	1.45	1.76	2.15	2.55	2.97	3	3.69	3.32
0.84	1.11	1.31	1.55	2.45	3.01	3.56	4.1	4.53	4.04
0.91	1.2	1.09	-3	2.48	3.53	4.21	4.93	5.5	4.88
1.1	1.46	1.79	2.24	3.42	4.2	4.97	5.85	6.68	5.84
1.06	1.41	1.7	2.14	3.89	4.9	5.85	6.92	8.15	6.94
0.92	1.18	0.7	-7.39	3.43	5.39	6.67	8.15	10	8.19
1.09	1.45	1.75	2.18	3.89	4.88	5.84	6.92	8.15	6.94
1.07	1.56	2.05	2.65	3.38	4.11	4.92	5.83	6.68	5.82

Potential-Based Reward Shaping

- $R'(s, a, s') = R(s, a, s') + \gamma\Phi(s') - \Phi(s)$
- any other transformation may yield sub optimal policies unless further assumptions are made about the underlying MDP

Is Exploration Important?

Montezuma's Revenge

Is Exploration Important?

Theory

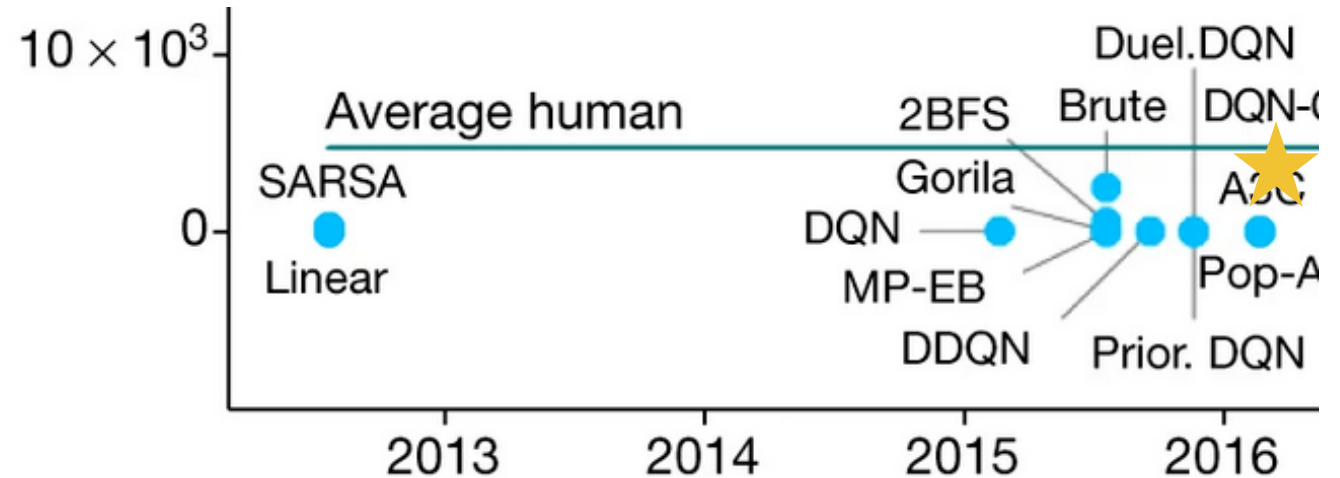
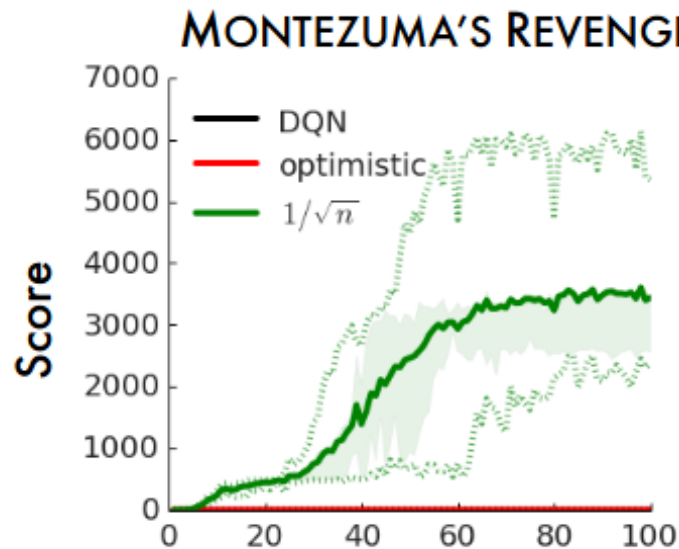
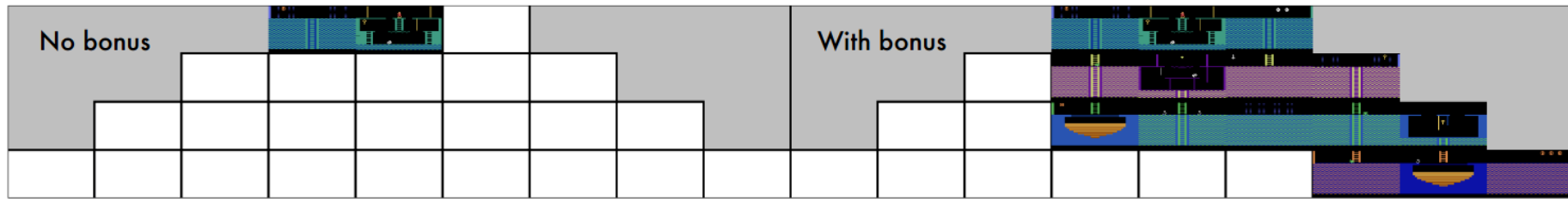
	Algorithm	Regret	Time	Space
Model-based	UCRL2 [10] ¹	at least $\tilde{\mathcal{O}}(\sqrt{H^4 S^2 AT})$	$\Omega(T S^2 A)$	$\mathcal{O}(S^2 AH)$
	Agrawal and Jia [1] ¹	at least $\tilde{\mathcal{O}}(\sqrt{H^3 S^2 AT})$		
	UCBVI [5] ²	$\tilde{\mathcal{O}}(\sqrt{H^2 SAT})$	$\tilde{\mathcal{O}}(T S^2 A)$	
	vUCQ [12] ²	$\tilde{\mathcal{O}}(\sqrt{H^2 SAT})$		
Model-free	Q-learning (ε -greedy) [14] (if 0 initialized)	$\Omega(\min\{T, A^{H/2}\})$	$\mathcal{O}(T)$	$\mathcal{O}(SAH)$
	Delayed Q-learning [25] ³	$\tilde{\mathcal{O}}_{S,A,H}(T^{4/5})$		
	Q-learning (UCB-H)	$\tilde{\mathcal{O}}(\sqrt{H^4 SAT})$		
	Q-learning (UCB-B)	$\tilde{\mathcal{O}}(\sqrt{H^3 SAT})$		
	lower bound	$\Omega(\sqrt{H^2 SAT})$	-	-

Table 1: Regret comparisons for RL algorithms on episodic MDP. $T = KH$ is totally number of steps, H is the number of steps per episode, S is the number of states, and A is the number of actions. For clarity, this table is presented for $T \geq \text{poly}(S, A, H)$, omitting low order terms.

Exploration Bonus

Example 1: Learn Pseudocount

$B(s, a) \approx \frac{1}{\sqrt{\hat{N}(s)}}$ where $\hat{N}(s)$ is a learned function approximation



Bellemare, et al. 2016 "Unifying Count-Based Exploration..."

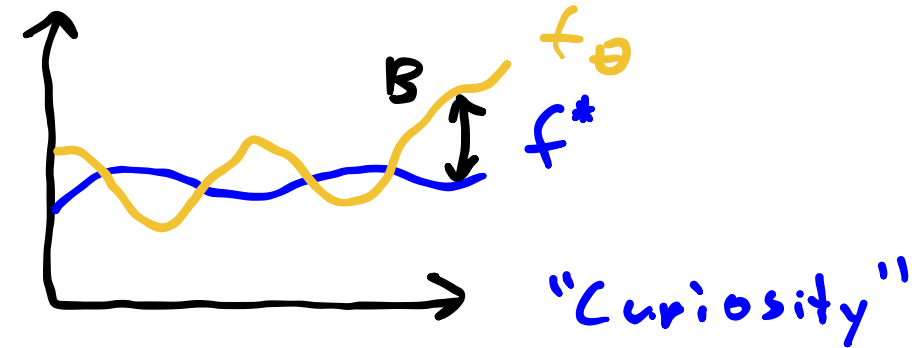
Exploration Bonus

Example 2: Learn a function of the state and action

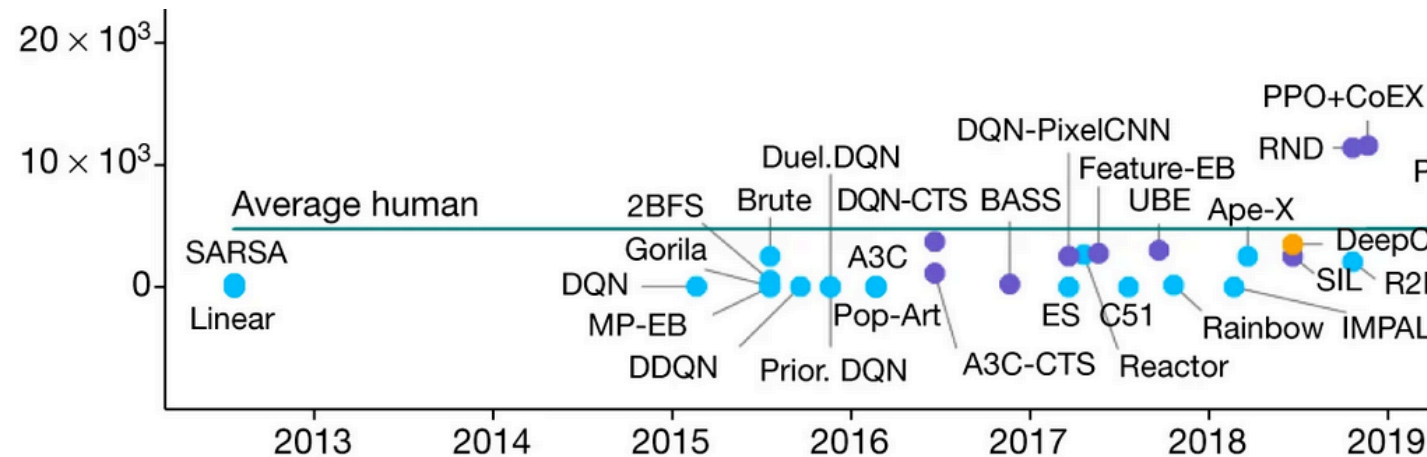
$$B(s, a) = \|\hat{f}_\theta(s, a) - f^*(s, a)\|^2$$

What should f^* be?

- $f^*(s, a) = s'$ (Next state prediction)
- $f^*(s, a) = f_\phi(s, a)$ where f_ϕ is a random neural network.



"Random Network Distillation"



Exploration Bonus

Example 3: Thompson Sampling

1. Maintain a distribution over Q ← Hard
2. Sample Q
3. Act according to Q

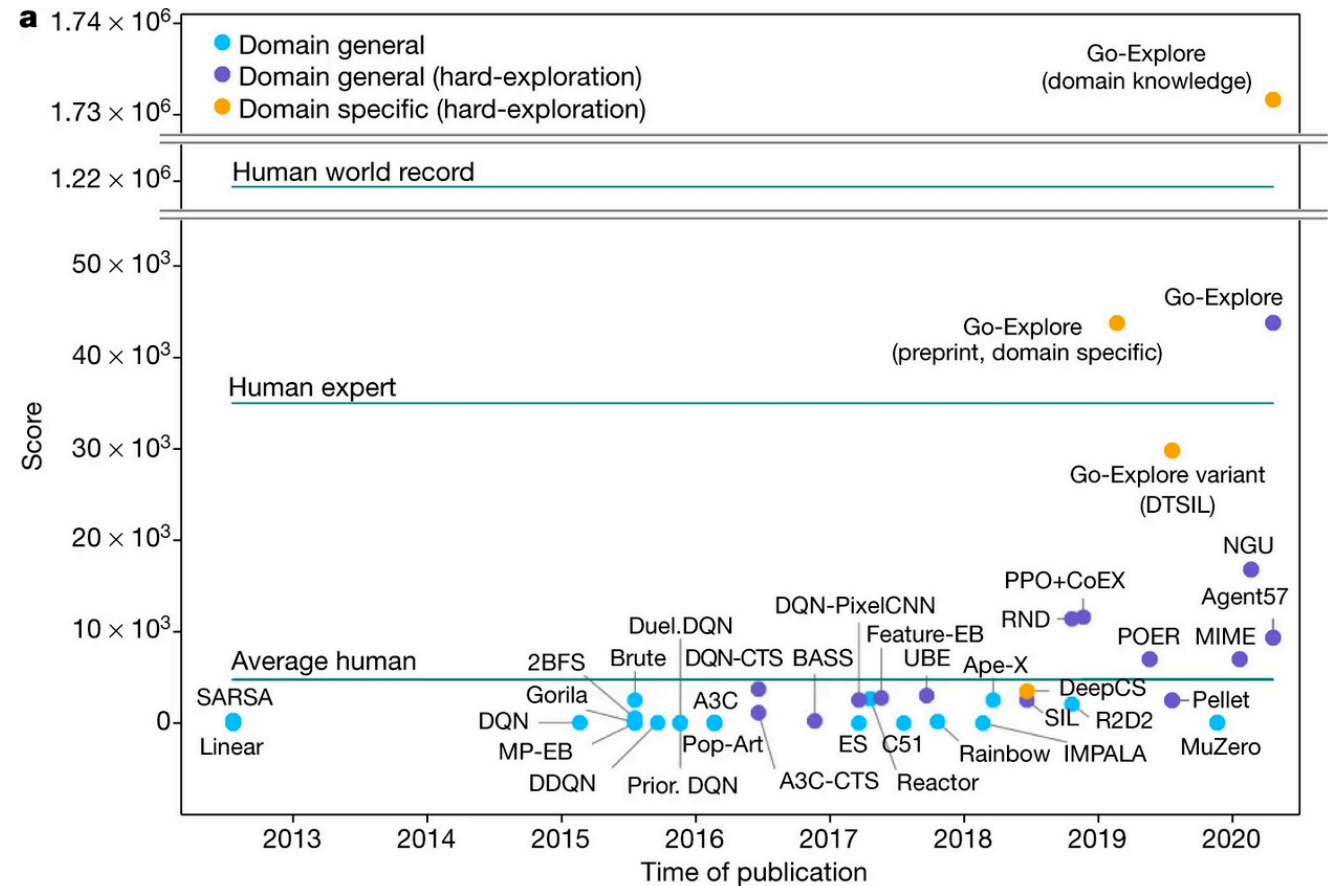
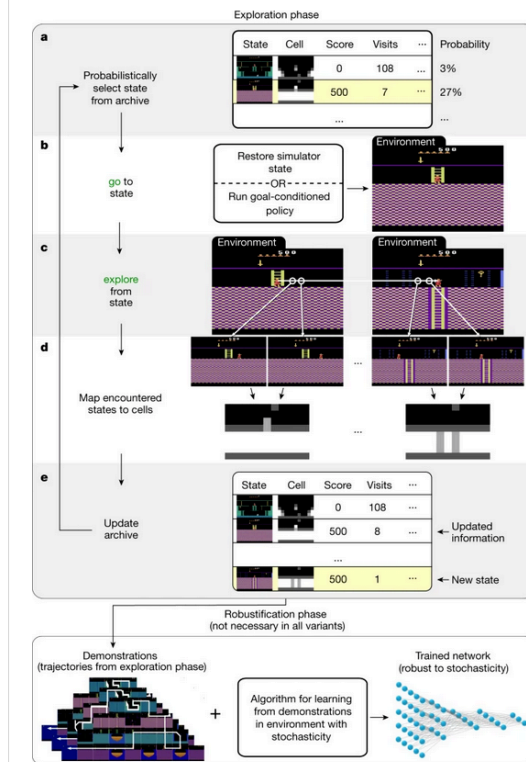
- Bootstrapping with multiple Q networks
- Dropout

Exploration Bonus

Example 4: Go-Explore

"First return, then explore"

Fig. 1: Overview of Go-Explore.



(Uber AI Labs)

Soft Actor Critic: Entropy Regularization

$$U(\pi) = E \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha \mathcal{H}(\pi(\cdot | s_t))) \right]$$

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi} [Q(\mathbf{s}_t, \mathbf{a}_t) - \log \pi(\mathbf{a}_t | \mathbf{s}_t)]$$

$$\mathcal{T}^{\pi} Q(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V(\mathbf{s}_{t+1})]$$

$$\pi_{\text{new}} = \arg \min_{\pi' \in \Pi} D_{\text{KL}} \left(\pi'(\cdot | \mathbf{s}_t) \parallel \frac{\exp(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot))}{Z^{\pi_{\text{old}}}(\mathbf{s}_t)} \right)$$

Soft Actor Critic

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors $\psi, \bar{\psi}, \theta, \phi$.

for each iteration **do**

for each environment step **do**

$$\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$$

$$\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$$

end for

for each gradient step **do**

$$\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$$

$$\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i) \text{ for } i \in \{1, 2\}$$

$$\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$$

$$\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$$

end for

end for

$$J_V(\psi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[\frac{1}{2} (V_\psi(\mathbf{s}_t) - \mathbb{E}_{\mathbf{a}_t \sim \pi_\phi} [Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)])^2 \right]$$

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \hat{Q}(\mathbf{s}_t, \mathbf{a}_t))^2 \right]$$

$$\hat{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim p} [V_{\bar{\psi}}(\mathbf{s}_{t+1})]$$

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi_\phi(\cdot | \mathbf{s}_t) \parallel \frac{\exp(Q_\theta(\mathbf{s}_t, \cdot))}{Z_\theta(\mathbf{s}_t)} \right) \right]$$

Soft Actor Critic

Advantages:

- Stable
- Learns many near-optimal policies
- Exploration
- Insensitivity to hyperparameters
- Off-Policy

Disadvantages

- Sensitive to α Solution = Entropy
constraint and adjust α

Wisdom

Deep RL: The Dream

openai / gym

Watch 995

<> Code Issues 112 Pull requests 13 Actions Projects Wiki Security Insights

Leaderboard

Aman Arora edited this page 3 days ago · 352 revisions

This page tracks the performance of user algorithms for various tasks in gym. Previously, users could submit their scores directly to gym.openai.com/envs, but it has been decided that a simpler wiki might do this task more efficiently.

This wiki page is a community driven page. Anyone can edit this page and add to it. We encourage you to contribute and modify this page and add your scores and links to your write-ups and code to reproduce your results. We also encourage you to add new tasks with the gym interface, but not in the core gym library (such as roboschool) to this page as well.

Links to videos are optional, but encouraged. Videos can be youtube, instagram, a tweet, or other public links. Write-ups should explain how to reproduce the result, and can be in the form of a simple gist link, blog post, or github repo.

We have begun to copy over the previous performance scores and write-up links over from the [previous page](#). This is an ongoing effort, and we can use some help.

Environments

Classic control

CartPole-v0

A pole is attached by an un-actuated joint to a cart, which moves along a frictionless track. The system is controlled by applying a force of +1 or -1 to the cart. The pendulum starts upright, and the goal is to prevent it from falling over. A reward of +1 is provided for every timestep that the pole remains upright. The episode ends when the pole is more than 15 degrees from vertical, or the cart moves more than 2.4 units from the center.

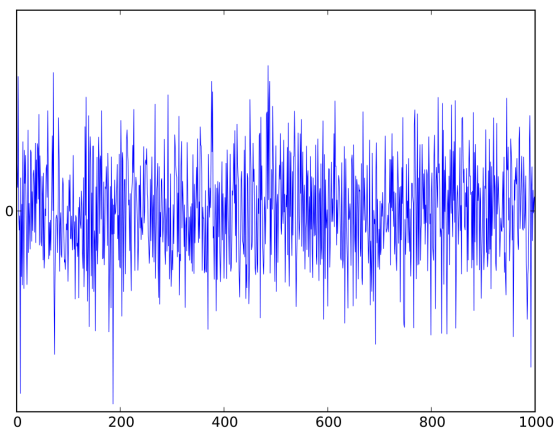
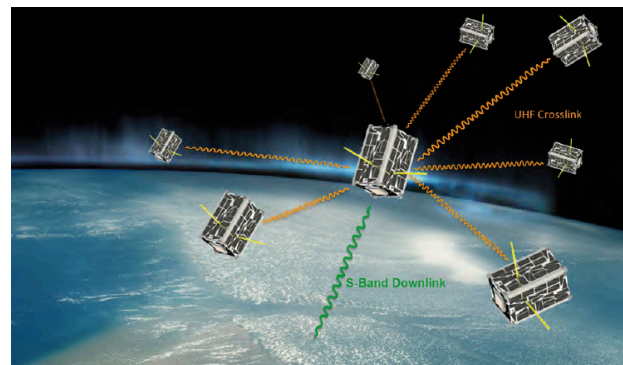


- [Environment Details](#)
- CartPole-v0 defines "solving" as getting average reward of 195.0 over 100 consecutive trials.*
- This environment corresponds to the version of the cart-pole problem described by Barto, Sutton, and Anderson [Barto83].*

User	Episodes before solve	Write-up	Video
Zhiqing Xiao	0 (use close-form preset policy)	writeup	
Hengjian Jia	0 (use close-form PID policy)	code/writeup	
Keavnn	0	writeup	
Shakti Kumar	0	writeup	Video
Nextgrid.ai 🍌	0	writeup	Video
iRyanBell	2	writeup	

Using Deep RL for your problem

1. Some interesting problem (smallsat swarm)
2. Spend weeks theorizing about the exact-right cost function and dynamics
3. Decide RL can solve all of your problems
4. Fire up open-ai baselines
5. Does it work??

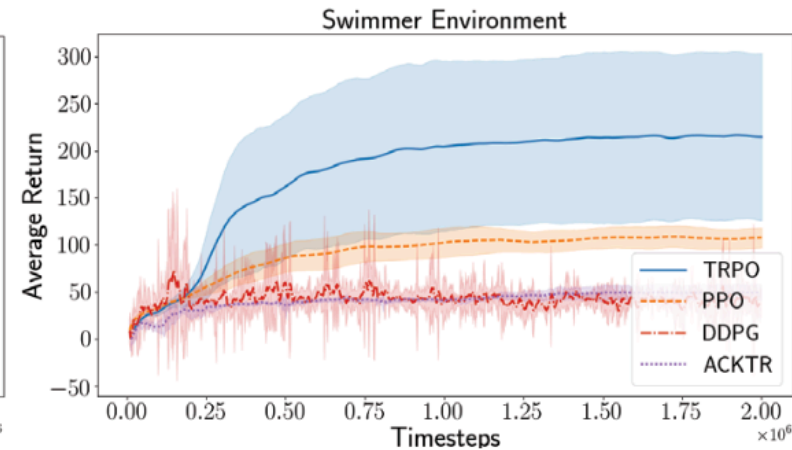
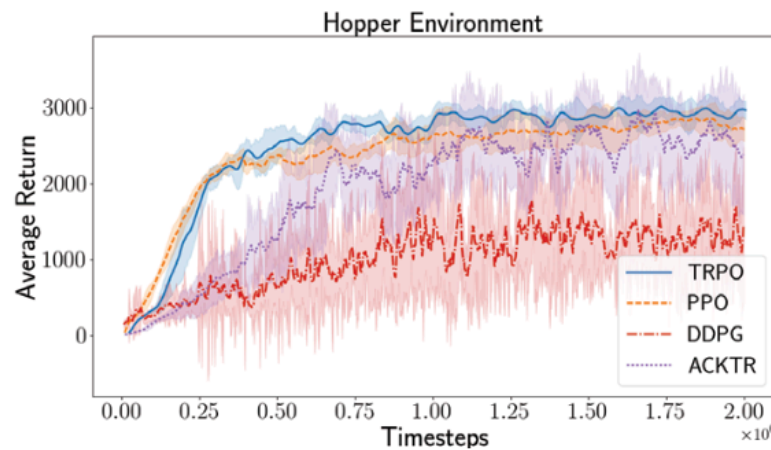
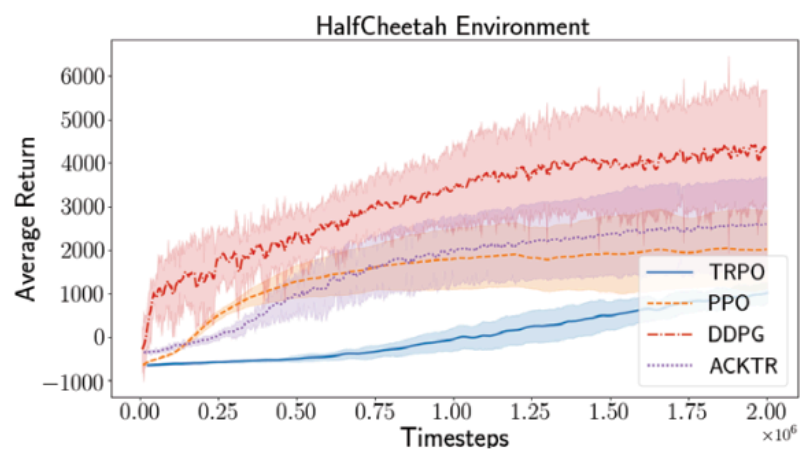


 [openai / baselines](#)

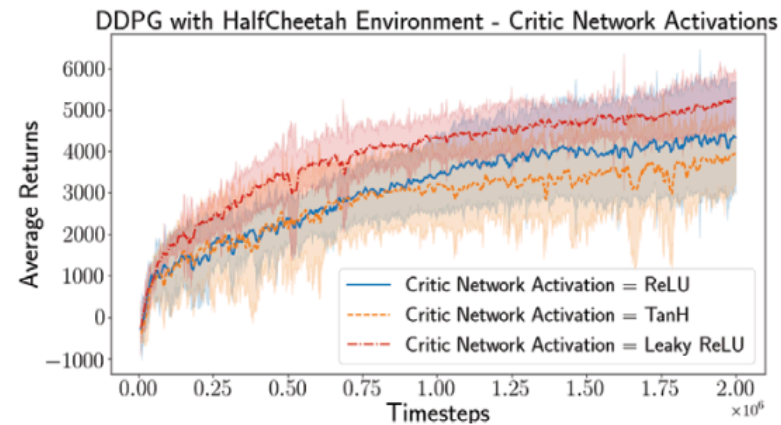
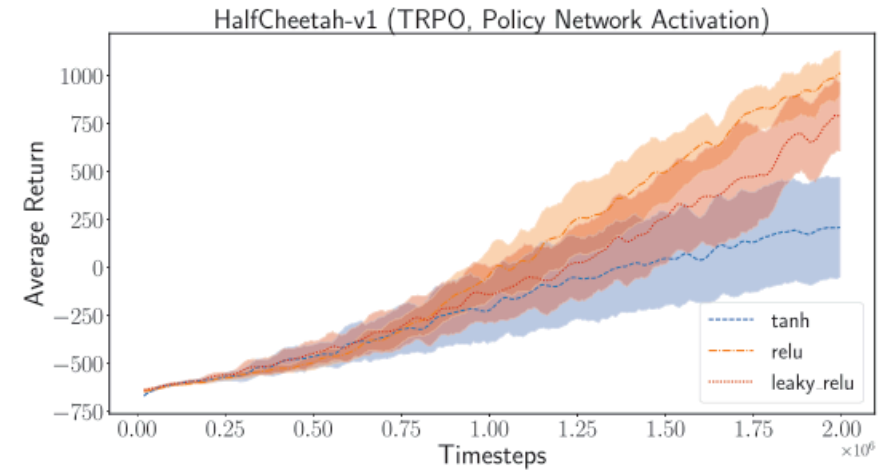
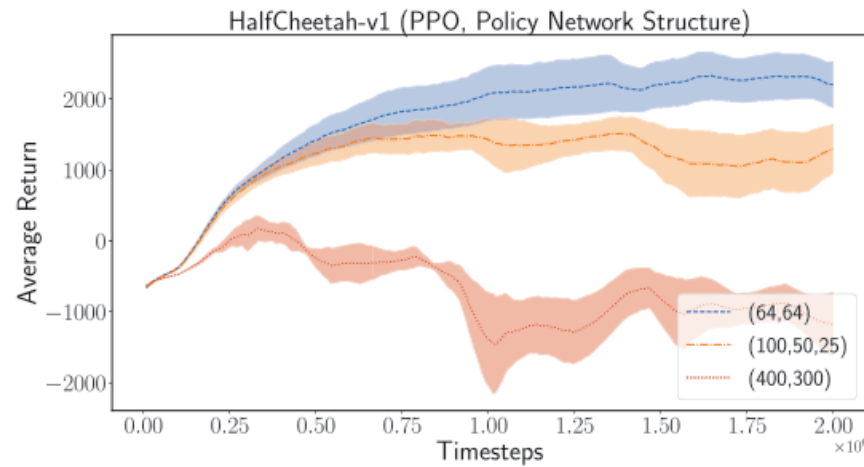
Why not?

- Hyperparameters?
- Reward scaling?
- Not enough training time????

Algorithms

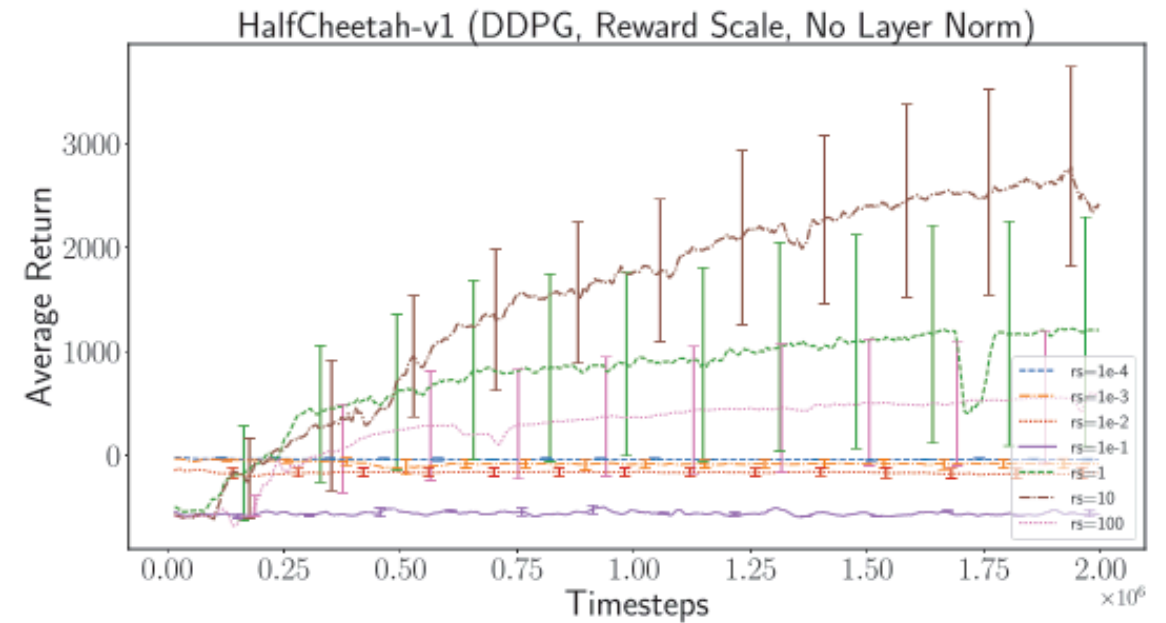
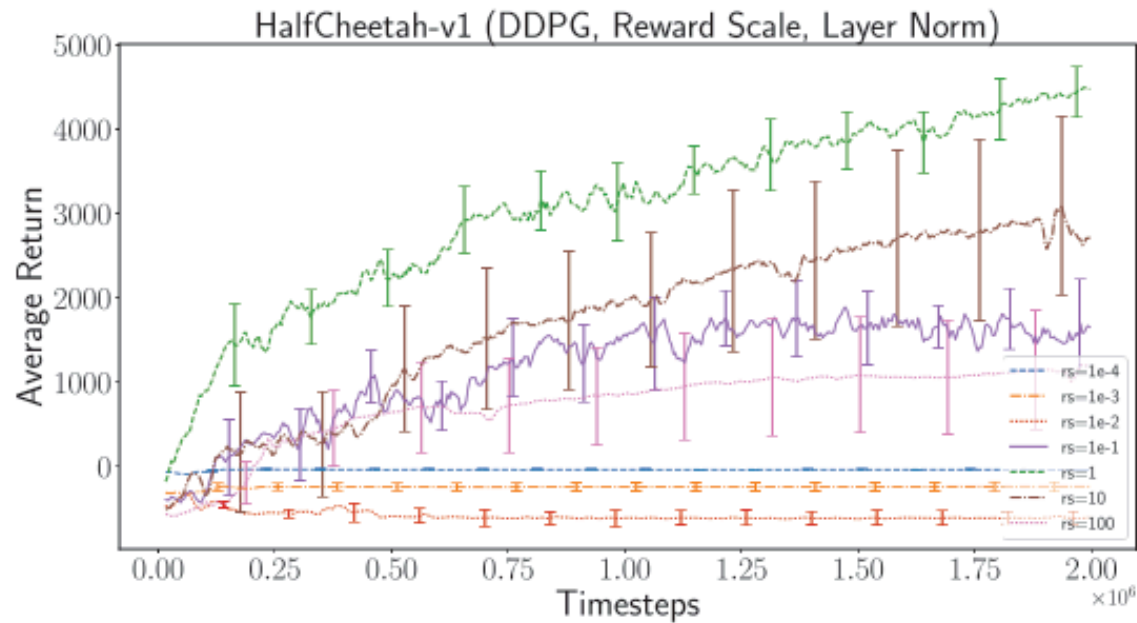


Policy Network Architecture



Reward Rescaling

"simply multiplying the rewards generated from an environment by some scalar"



Statistical Significance

"Unfortunately, in recent reported results, it is not uncommon for the top-N trials to be selected from among several trials (Wu et al. 2017; Mnih et al. 2016)"

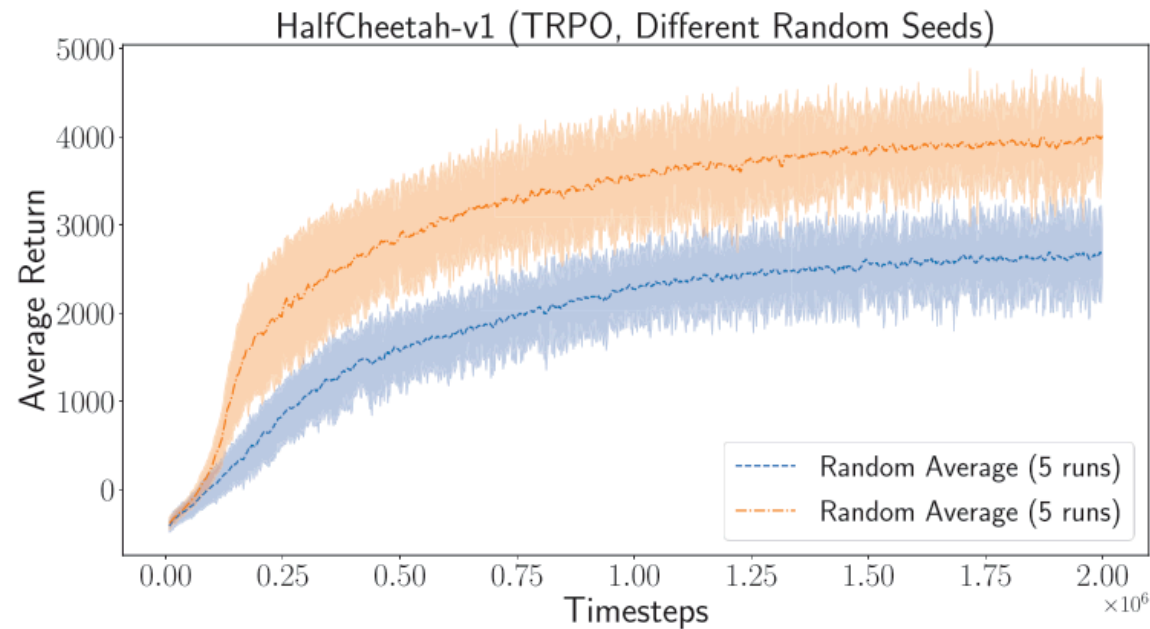
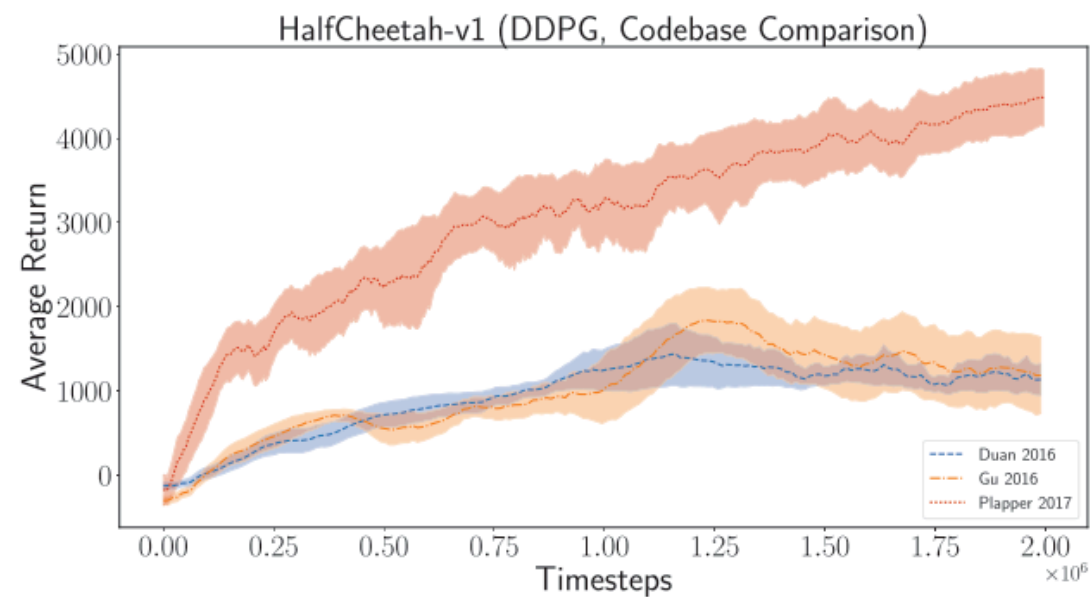
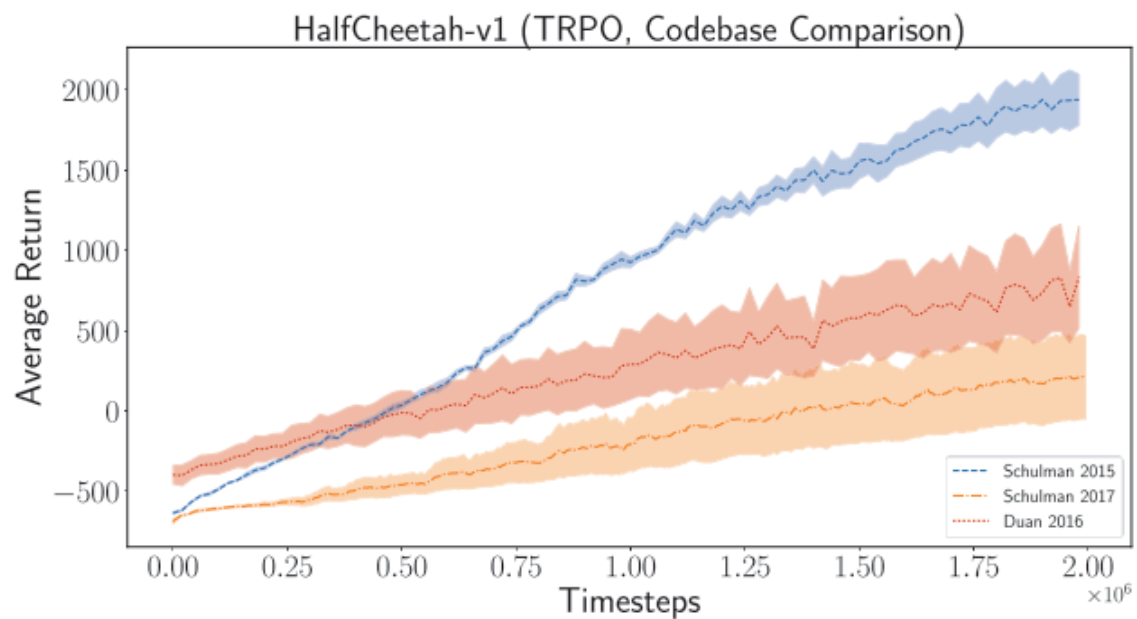


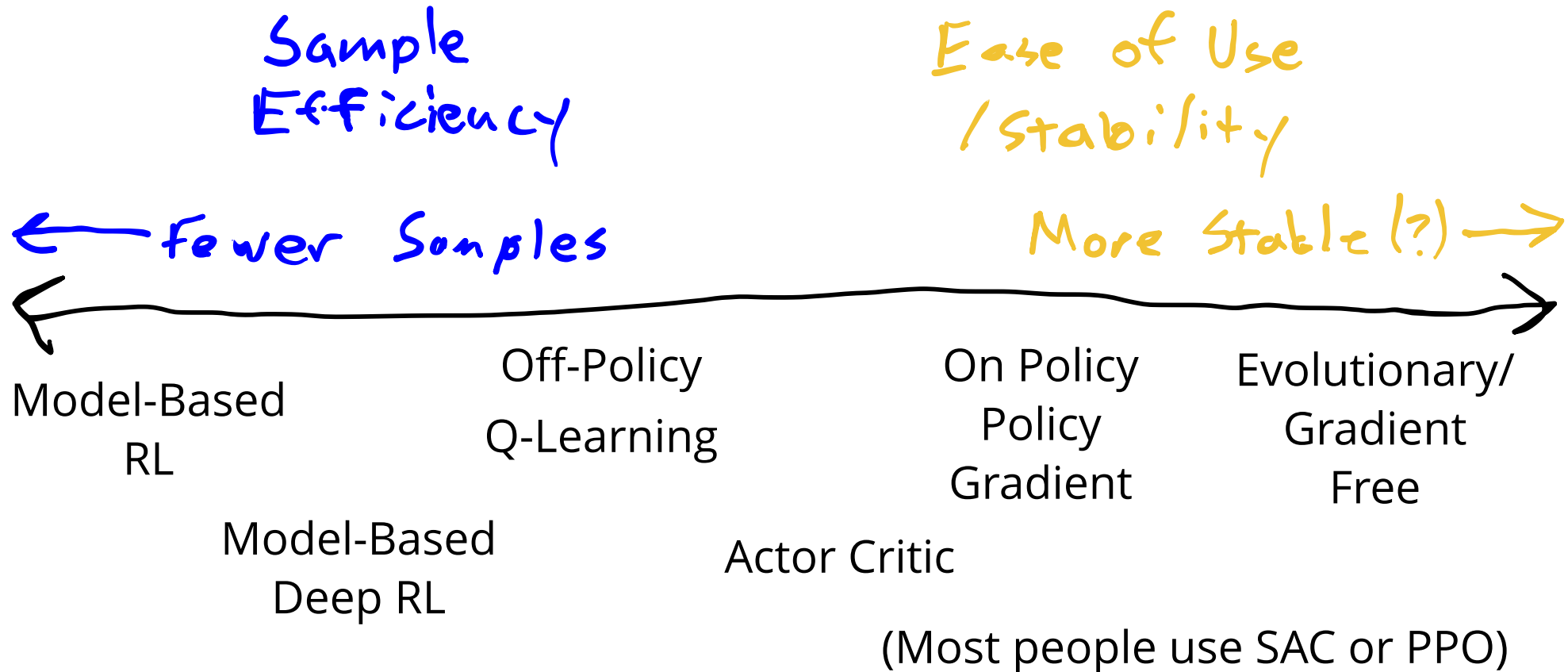
Figure 5: TRPO on HalfCheetah-v1 using the same hyperparameter configurations averaged over two sets of 5 different random seeds each. The average 2-sample t -test across entire training distribution resulted in $t = -9.0916$, $p = 0.0016$.

Codebases



How to choose an RL Algorithm

(According to Sergey Levine)



How to be successful with RL

- Always **start with a small problem that works** and scale up (keep verifying that it works with every change)
- Plot everything that you can think of (TensorBoard)
 - *Losses*
 - Policies
 - Value functions
 - Trajectories
 - (Average return) Learning curve
- Keep calm and lower your learning rate

